Name : Neil Taison RIGAUD

Student ID : 411221367

Subject : Intro to Image Processing – Assignment #2

Date : 2025/10/22

1- Code Listing and Explanation

```python
import numpy as np
import matplotlib.pyplot as plt
from PIL import Image
```

We import the necessary libraries for:

- Opening and saving images:
  - ***Image*** from the ***PIL*** library
- Processing the pixel values from the opened images:
  - ***numpy***
- Creating the histograms:
  - ***pyplot*** from ***matplotlib***

```python
# Constants
RGB_WEIGHTS = [0.299, 0.587, 0.114]
IMG_TITLE = "bear.jpg"
OUTPUT_TITLE = "bear.jpg"
```

We define the weights for converting the color image to a grayscale using the given formula in the previous assignment along with constant values for opening and saving image file outputs.

```python
def color_to_gray(img_title, output_title):
    img = Image.open(img_title)

    # Get the pixels data
    pixels = np.array(img)

    # If image has alpha channel, ignore it
    if pixels.ndim == 3 and pixels.shape[2] >= 3:
        # Compute weighted sum across color channels -> single channel gray
        # Use dot product with RGB_WEIGHTS and clip to uint8
        gray = np.dot(pixels[..., :3], np.array(RGB_WEIGHTS))
    else:
        # Already single channel
        gray = pixels.astype(float)

    # Convert to uint8
    gray_uint8 = np.clip(gray, 0, 255).astype(np.uint8)

    # Create a 3-channel image where all channels equal gray for saving
    gray_rgb = np.stack([gray_uint8] * 3, axis=-1)

    # Save the output grayscale image (visualized as RGB where channels equal)
    out_img = Image.fromarray(gray_rgb)
    out_img.save(output_title)

    return gray_uint8
```

Here we proceed to convert the color image to a grayscale image by using the conversion formula with the previously defined weights to give each of the RGB channels the same weighted sum value for each pixels in the image.

```python
def count_gray_levels(pixels, range_max=256):
    """Count the gray level values in an array of pixels values."""
    # Flatten and use numpy bincount for speed and correctness
    flat = pixels.ravel()
    obs = np.bincount(flat, minlength=range_max)
    return obs
```

This function is used to count the number of observed gray level values in the image for the range 0 ~ 255.

```python
def histogram_equalization(gray_levels):
    """
    Given gray_levels (counts per original gray level 0..L-1),
    return:
      - equalized_counts: counts per equalized gray level (0..L-1) after mapping
      - mapping: array of length L where mapping[k] is the new level for original level k
    """
    L = len(gray_levels)
    cumsum = np.cumsum(gray_levels)
    n = cumsum[-1]

    # Avoid division by zero
    if n == 0:
        return np.zeros(L), np.arange(L)

    # mapping (s[k]) = round((L-1)/n * cumulative_sum[k])
    mapping = np.round((L - 1) * cumsum / n).astype(int)

    # Aggregate pixel counts into mapped bins
    equalized_counts = np.bincount(mapping, weights=gray_levels, minlength=L)

    return equalized_counts, mapping
```

This is the core function for the histogram equalization procedure where we start by computing the **cumulative summation** for each gray level value **i**, then we round the values previously obtained into a ***mapping* of rounded values** that will be used for generating the enhanced image. Finally we count the respective observations for each of the distinct **rounded values of the cumulative summations** and store them into ***equalized_counts***. Finally we return those values for generating the resulting image and produce the equalized histogram.

```python
def create_barplot(obs, x_label, y_label, title, output_name, edge_color="black"):
    """Creates a barplot histogram for a list of observations."""

    # Increasing array for the x-axis
    x_values = np.arange(len(obs))

    # Plot the histogram
    plt.bar(
        x_values, obs, width=0.2, align="center", color="skyblue", edgecolor=edge_color
    )
    plt.xlabel(x_label)
    plt.ylabel(y_label)
    plt.title(title)
    plt.savefig(output_name, dpi=300, bbox_inches="tight")
    plt.close()
```

This function helps create the different barplots that we need before and after the histogram equalization process. As mentioned earlier, we use ***pyplot*** to plot the ***discrete histogram*** from the array of gray level observations and save the resulting image using the name parameter passed.

```python
def apply_mapping(gray_pixels, mapping, output_title):
    """Apply the histogram-equalization mapping to a 2D gray image and save result.

    mapping: 1D array where mapping[old_level] = new_level
    Returns the mapped 2D uint8 image.
    """
    # Ensure mapping can index up to 255
    mapping = np.asarray(mapping).astype(np.uint8)

    # Apply mapping using vectorized indexing
    mapped = mapping[gray_pixels]

    # Save as 3-channel RGB for viewing
    mapped_rgb = np.stack([mapped] * 3, axis=-1)
    out_img = Image.fromarray(mapped_rgb)
    out_img.save(output_title)

    return mapped
```

This function uses a mapping generated from the computation of the **rounded cumulative sum** of the gray level values from the **Histogram Equalization** procedure and generates the resulting enhanced image after the transformation.

```python
if __name__ == "__main__":
    pixels = color_to_gray(IMG_TITLE, OUTPUT_TITLE)
    gray_level_counts = count_gray_levels(pixels)

    create_barplot(
        gray_level_counts,
        "Gray level value",
        "Observations",
        "Histogram BEFORE Equalization",
        "before_equalization.png",
        "blue",
    )

    equalized_counts, mapping = histogram_equalization(gray_level_counts)
    create_barplot(
        equalized_counts,
        "Gray level value",
        "Observations",
        "Histogram AFTER Equalization",
        "after_equalization.png",
        "red",
    )

    # Apply mapping to produce equalized image
    equalized_img = apply_mapping(pixels, mapping, "lena_equalized.png")
```

Finally, we have the main program procedure which uses the previously defined functions to:

1. Convert the original color image into a grayscale image using the ***color_to_gray*** function.
2. Obtain the gray level values for creating the original histogram for the converted grayscale image using the ***count_gray_levels*** function.
3. Create the histogram **before** equalization using ***create_barplot***
    a. Note: We use *barplot* in the function name because we are using the *discrete* probability distribution.
4. Obtain the equalized gray level count values and mapping to generate the enhanced image and final equalized histogram.
5. Create the histogram **after** equalization using the equalized gray level values.

2- Result images and corresponding histograms